

The Beginning Of A Journey To The Edge

Cloud Native Latvia

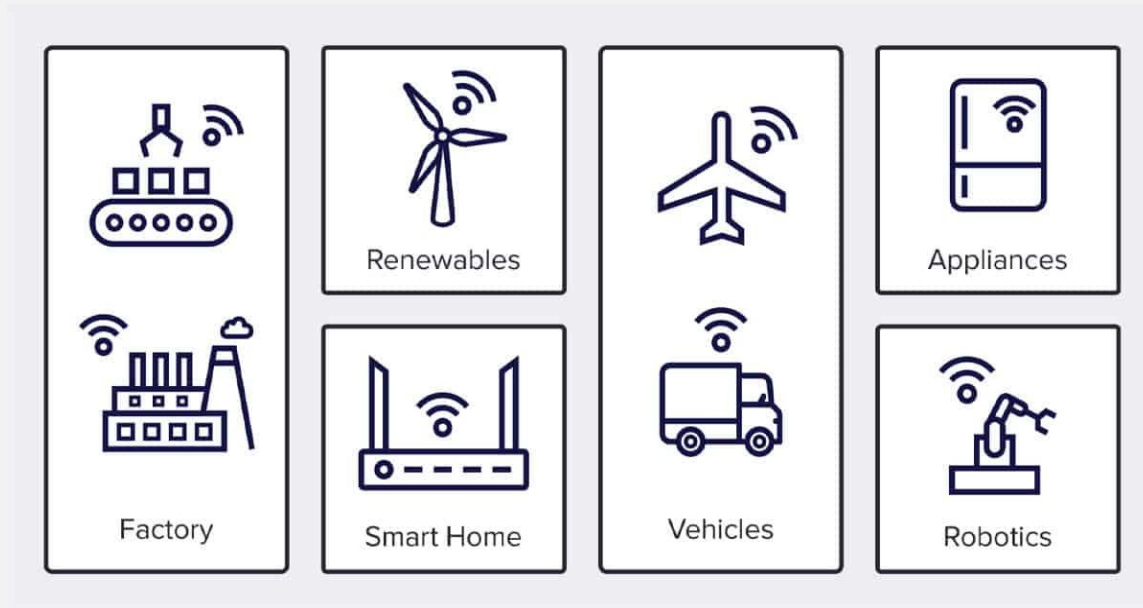
Meetup 2026



Scandinavian Sleeping & Living

What does "edge computing" mean for you?

*"Edge computing is a distributed computing model that brings data processing and storage **closer** to the **physical location** where data is generated or consumed, rather than relying on centralized data centers."* - Wikipedia



Khaled Hammoun

Platform Engineer, Container Platform Engineering

Daniel Hansen

Platform Engineer, Container Platform Engineering



A family-owned business

JYSK is an international and family-owned home furnishing retailer with Scandinavian roots.

JYSK operates more than 3,500 physical stores and webshops in 48 countries.

JYSK employs 31,000 colleagues.





5,000
JYSK stores

I am proud that JYSK now has more than 2,500 stores, but that does not mean that we are done.

We constantly need to grow, and the next long-term goal is to double the number of stores worldwide to 5,000.

Lars Larsen in 2017

3,591 JYSK stores

As of January 2026 (incl. webshops)

- **JYSK**
28 countries
3,374 stores
- **JYSK FRANCHISE**
21 countries
217 stores



IT in JYSK

IT Department

120+ Developers

Denmark, Germany, Poland, Portugal, Ukraine, Czech Republic, Austria, Portugal

60+ Operations & Infrastructure Engineers

Denmark, Poland

18 of which are **Platform Engineers** 🙌



IT Infrastructure

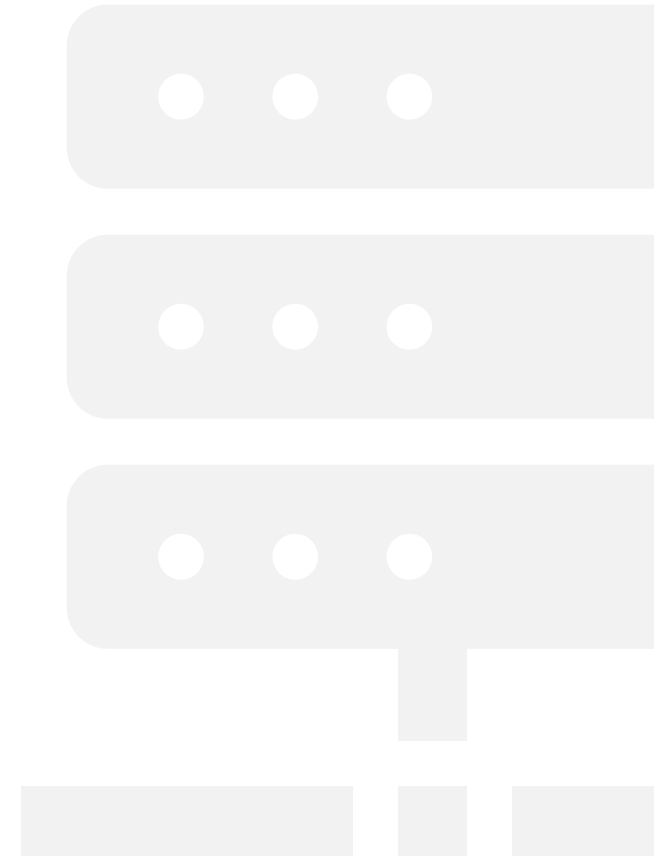
Occupy On-Premise, Edge, Cloud Installations

On-Premise

Over 4THz CPU, around 70TB RAM and 1.8PB storage
4000 virtual machines

In-Stores

5000 network switches – (think troubleshooting)
37.000 in-store devices, 11.000+ of which are mobile phones



Data Points

40 critical internal services power

- Webshops,
- Store Systems

6000+ requests/sec (internal systems)

200.000+ web requests/sec on edge

4000 Virtual machines

14 on-premise Kubernetes clusters, over 200+ worker nodes

3500+ Kubernetes edge clusters

1.42 trillion store metrics

1.5TB+ a day in logs

650.000+ store transactions a day



Fact: 290km of paper receipts... a day 🤖

The Beginning Of A Journey To The Edge



Wait.. Aren't we "On the edge" already?

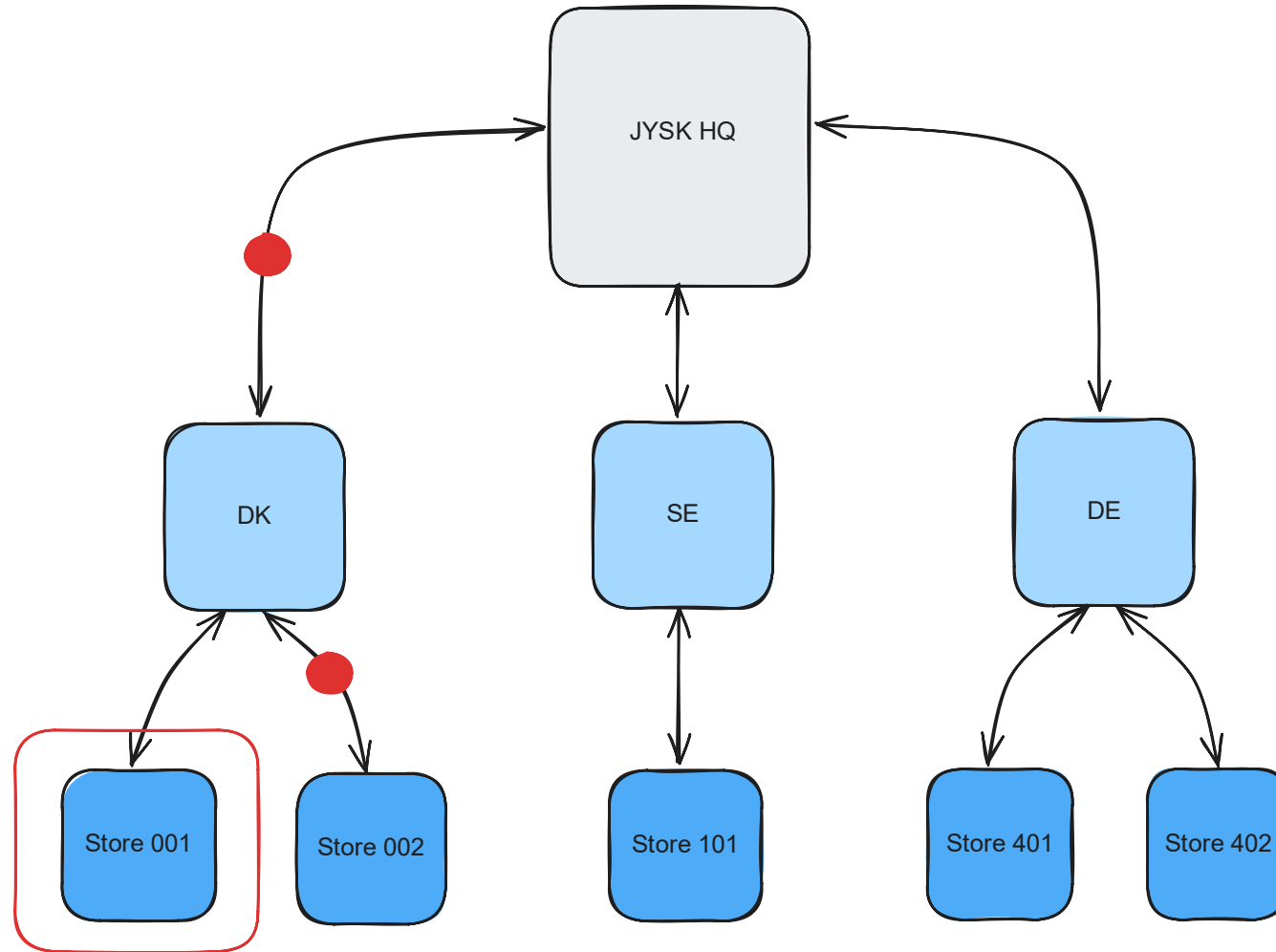
- 10 large distribution centers for **all** stores
- What about our stores?



The Problem



Stores and the Edge



Project Outline

- A solution that can give **value**
- Small project, that can get off the ground quickly
- Extensible
- Alignment possibilities of our future “generic platform”
- Business motivation

Requirements

- Data processing closer to its source
- Data-driven applications
- Variation to the standards



The Driver

Global Bandwidth Monitoring



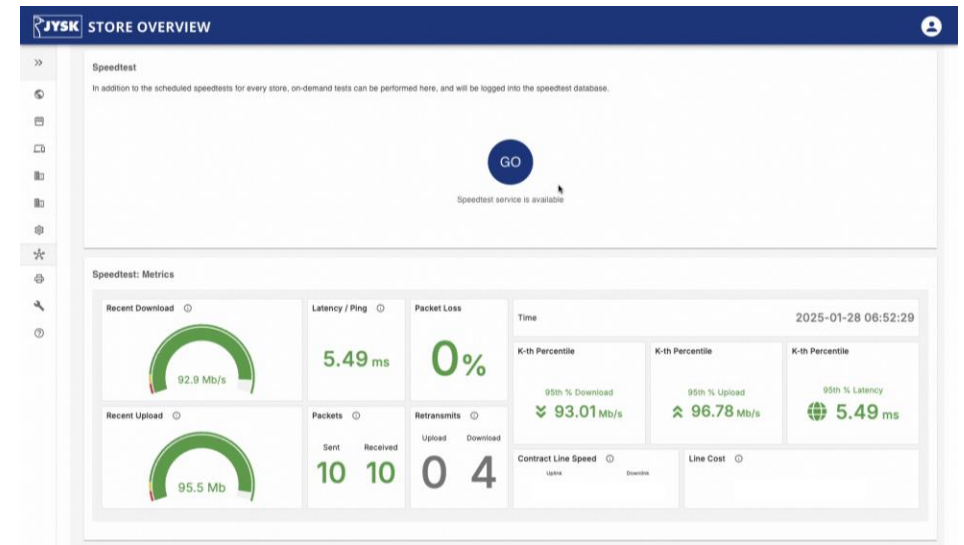
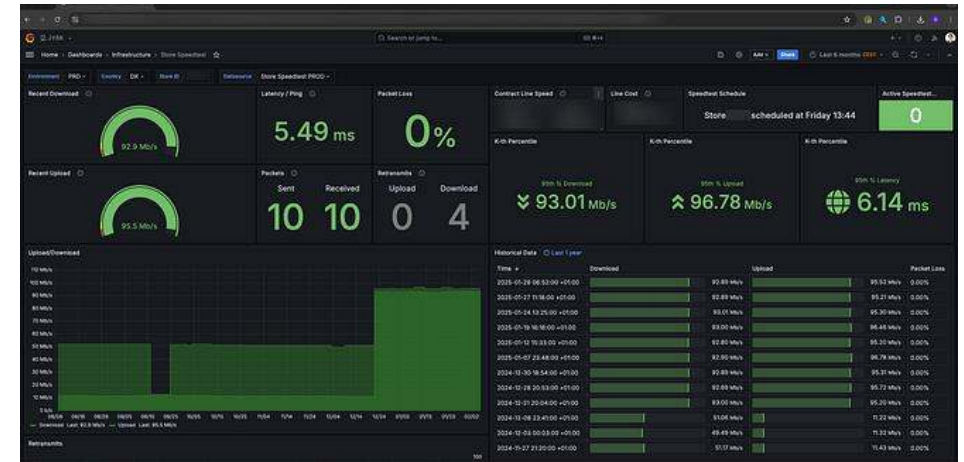
The Problem

- Current speed-test is a single batch file ran ad-hoc
- Event-driven, not data-driven
- Historical data is not possible
- Requires user knowledge to be able to report issues
- No idea if we are paying for something we are not using
 - or paying for something we cannot use!
- Can we handle the future?
 - Rollout of Unified Commerce
 - Supporting growth, more and more stores



The Solution

- Global bandwidth monitoring
- Ran at a schedule, for every store
- Historical data viewable
- Ad-hoc testing from a simple platform
- Easy deployed across 3500 stores
- Extensible



Kubernetes at the Edge



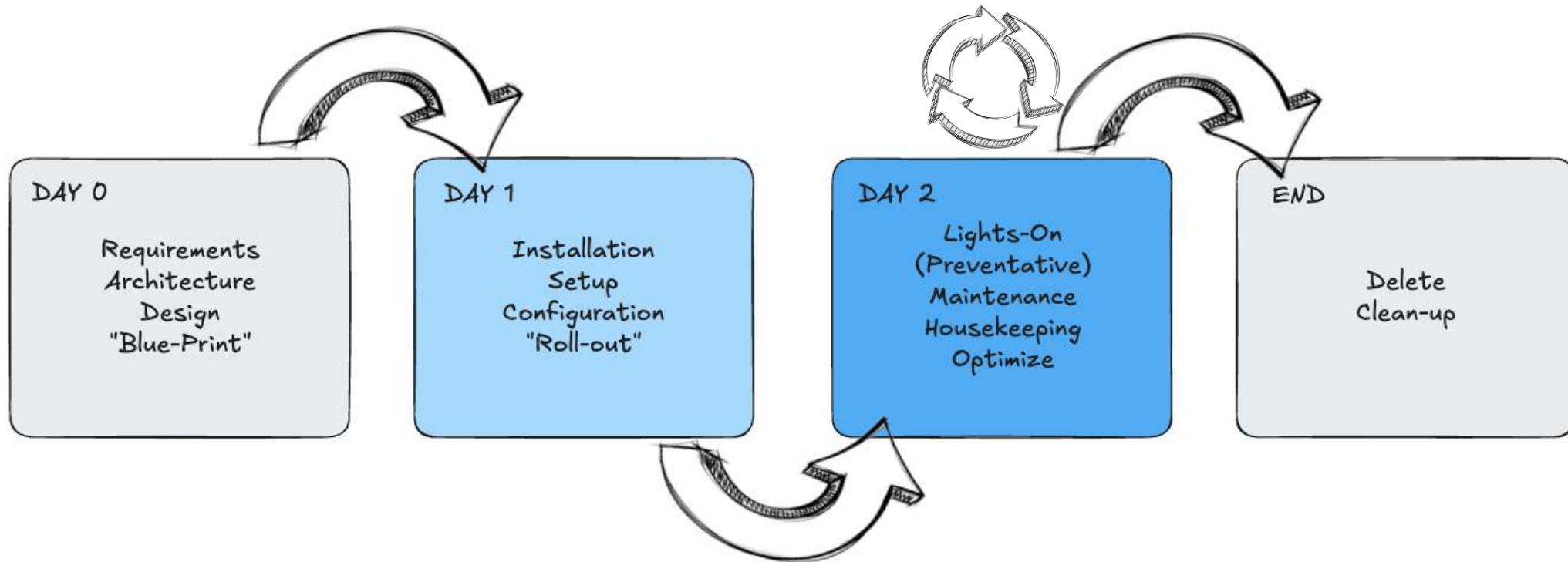
The Journey So Far

- Design and requirements
- The "Driver"
- **PoC with K3s**
- Alternatives?
 - k0s, k3s, Suse Edge Suite, OpenYurt(?), KubeEdge,
- What about operational disciplines (day 0,1,2)



Operational Phases

By design, edge compute is essentially unstable



What is Talos?

Talos Linux

Everything is an API



Built for Kubernetes



What is Talos for us?

- 3500 Kubernetes Nodes
- Disposable assets – cattle not pets
- Platform 2.0 “ready”
- Extensible

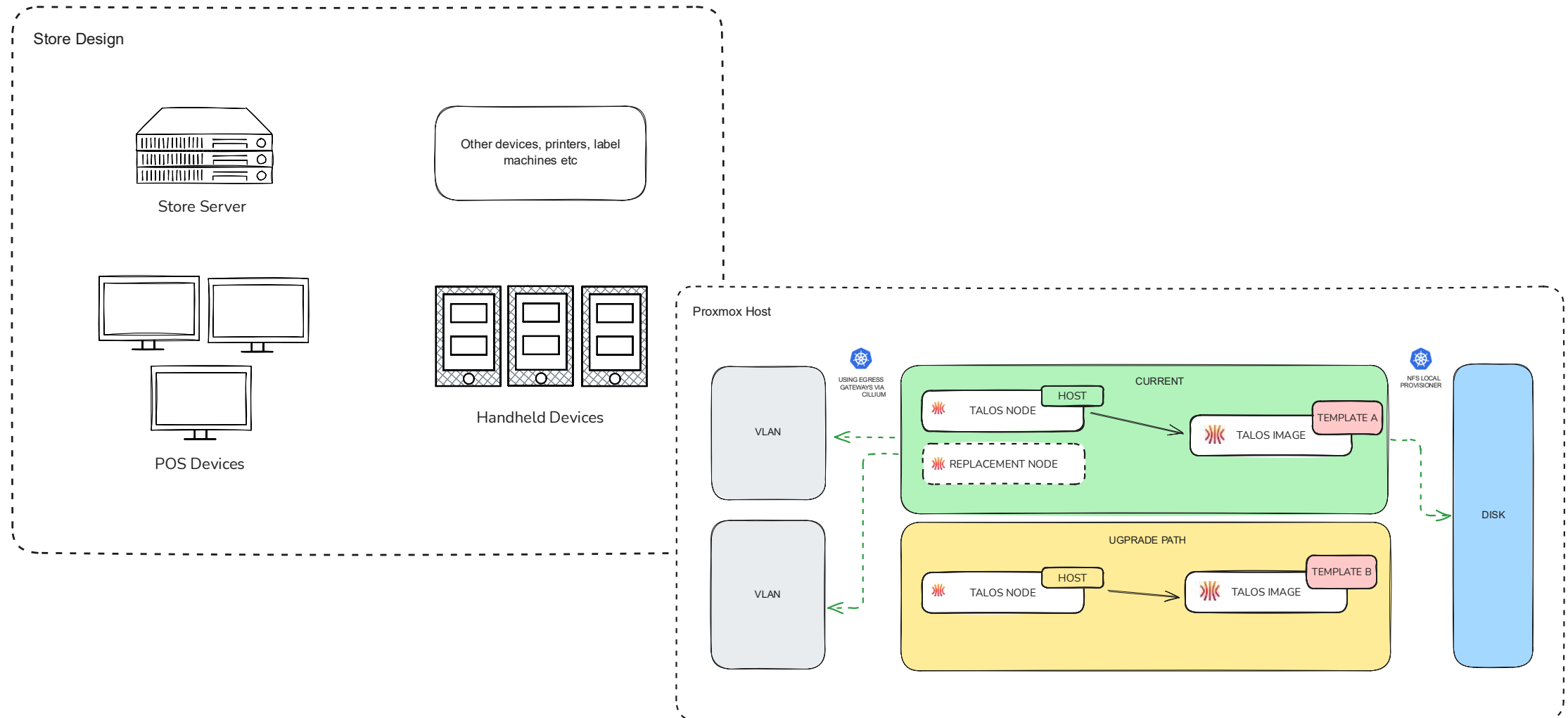
What is Talos for us?

- K3S Solution was valid, however!
- Ease of rollout
- Adoption is easy, we rolled our own backend
- Community is strong

The Design



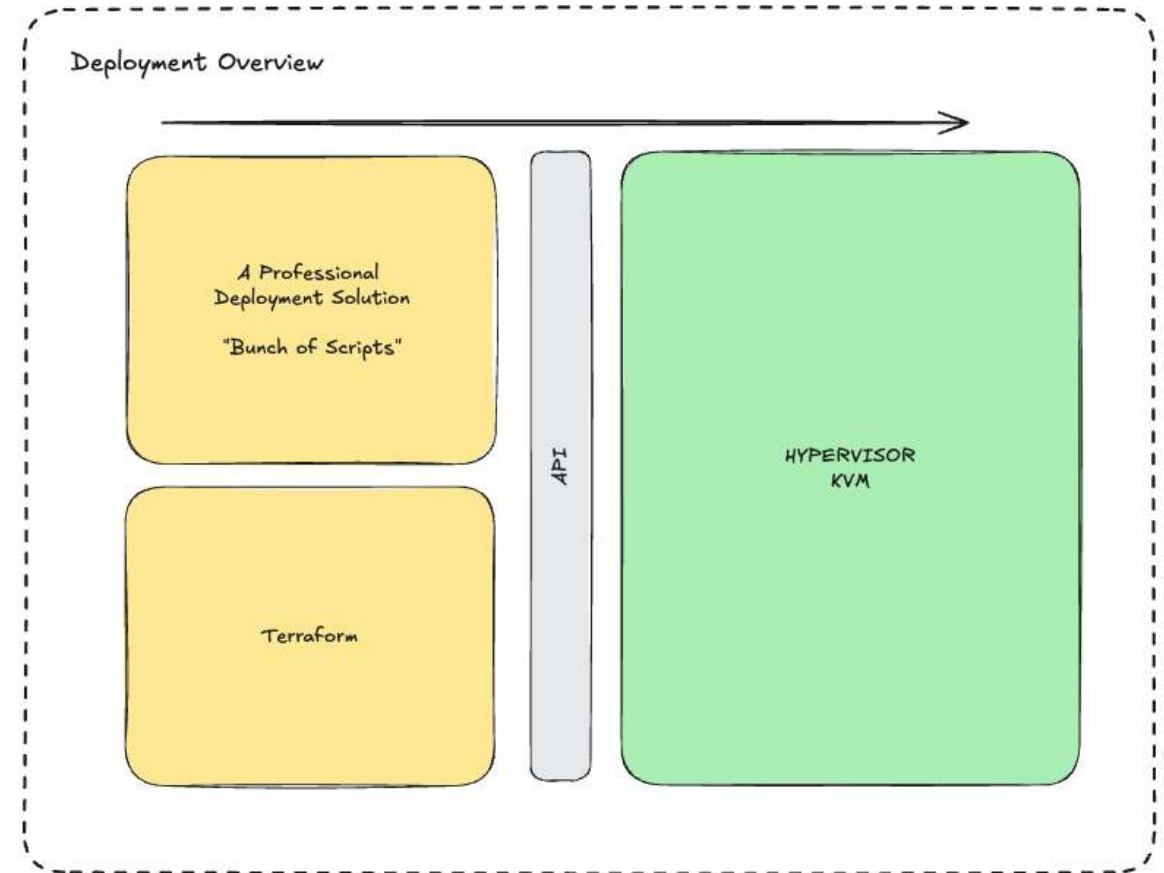
Landscape and Deployment Design



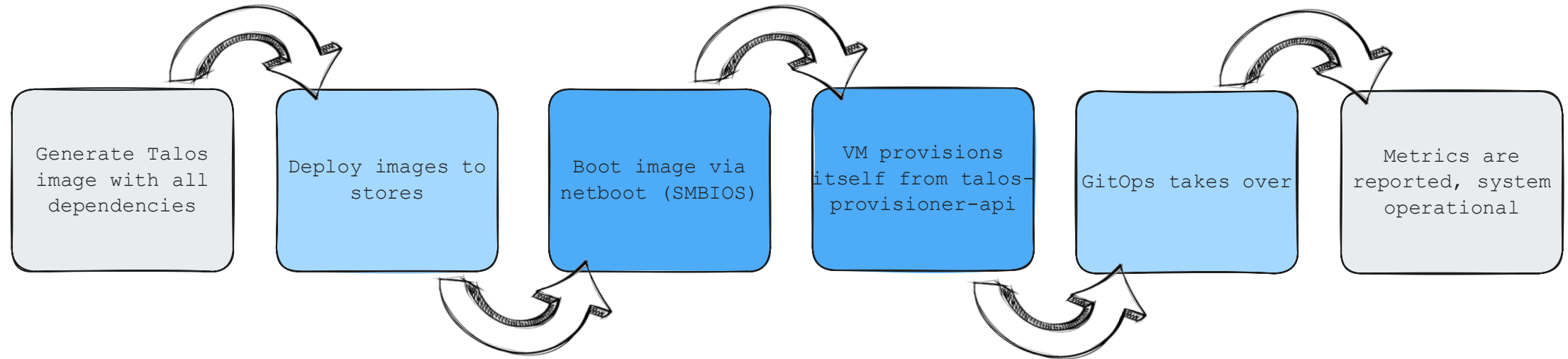
Deployment Idea

We need a simple deployment mechanism

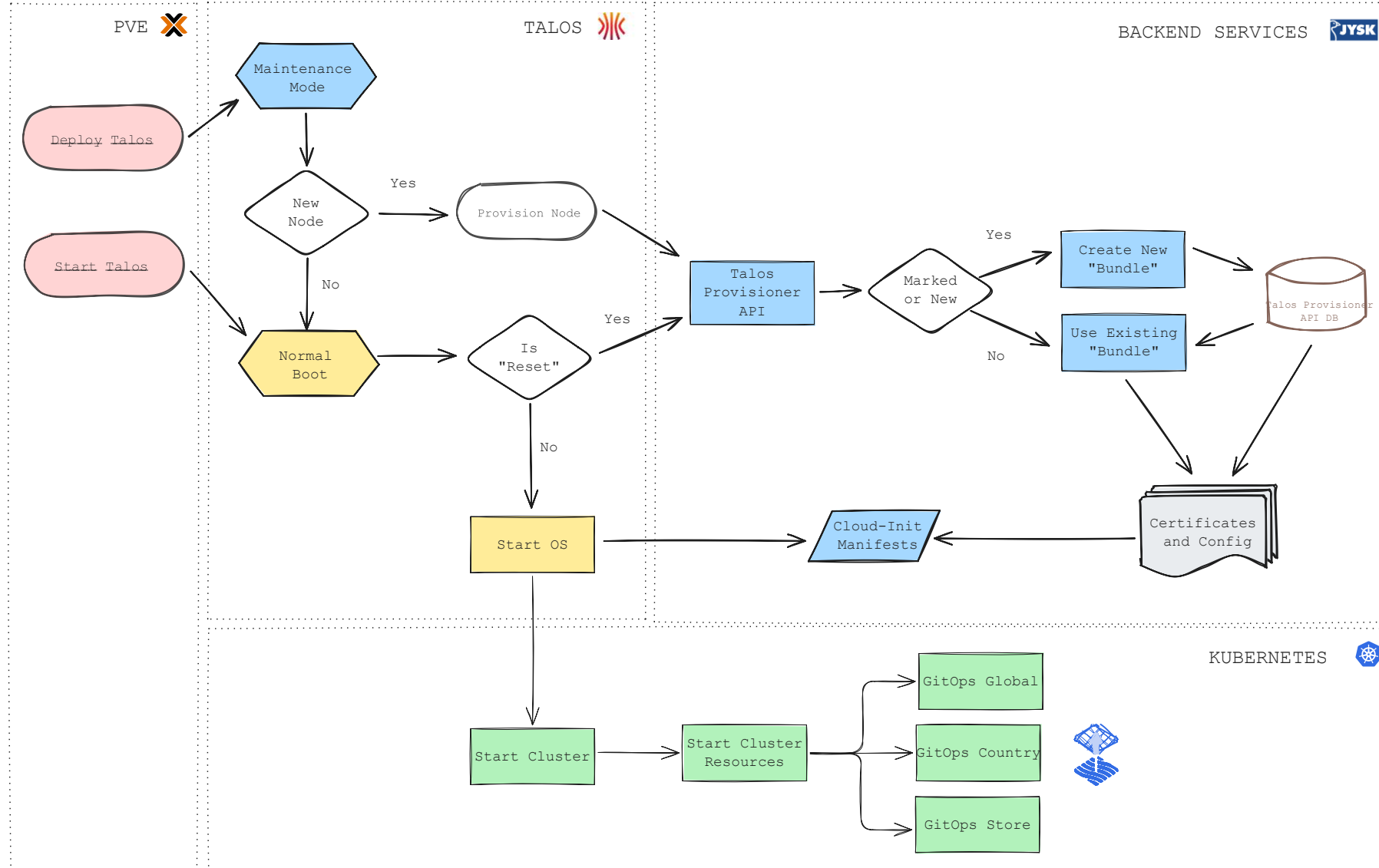
- Uses the PVE API via qm cli
- Can run batch jobs
- Extensible
 - We need to be able to deploy at scale, safely.



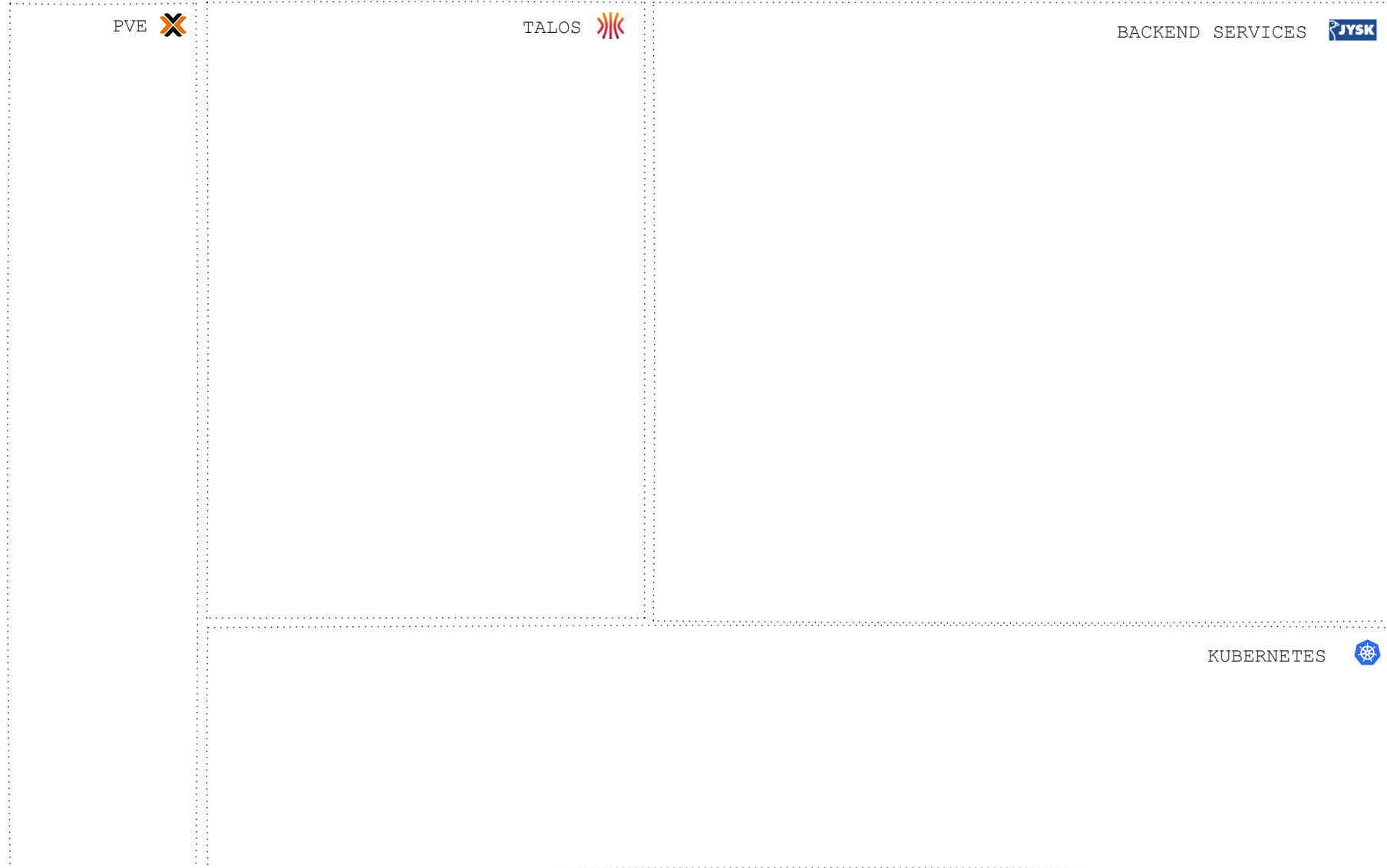
Deployment



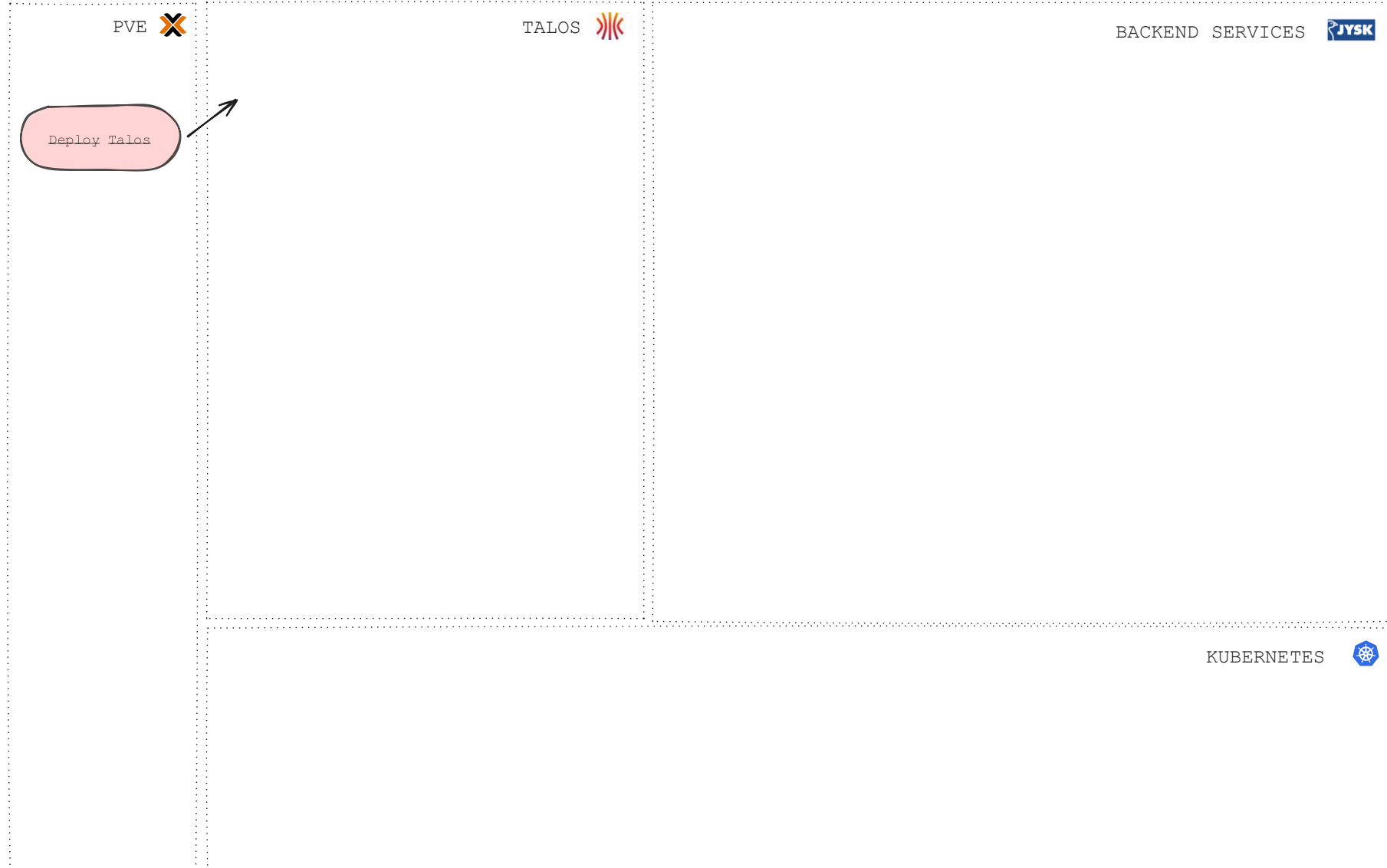
Deployment Process



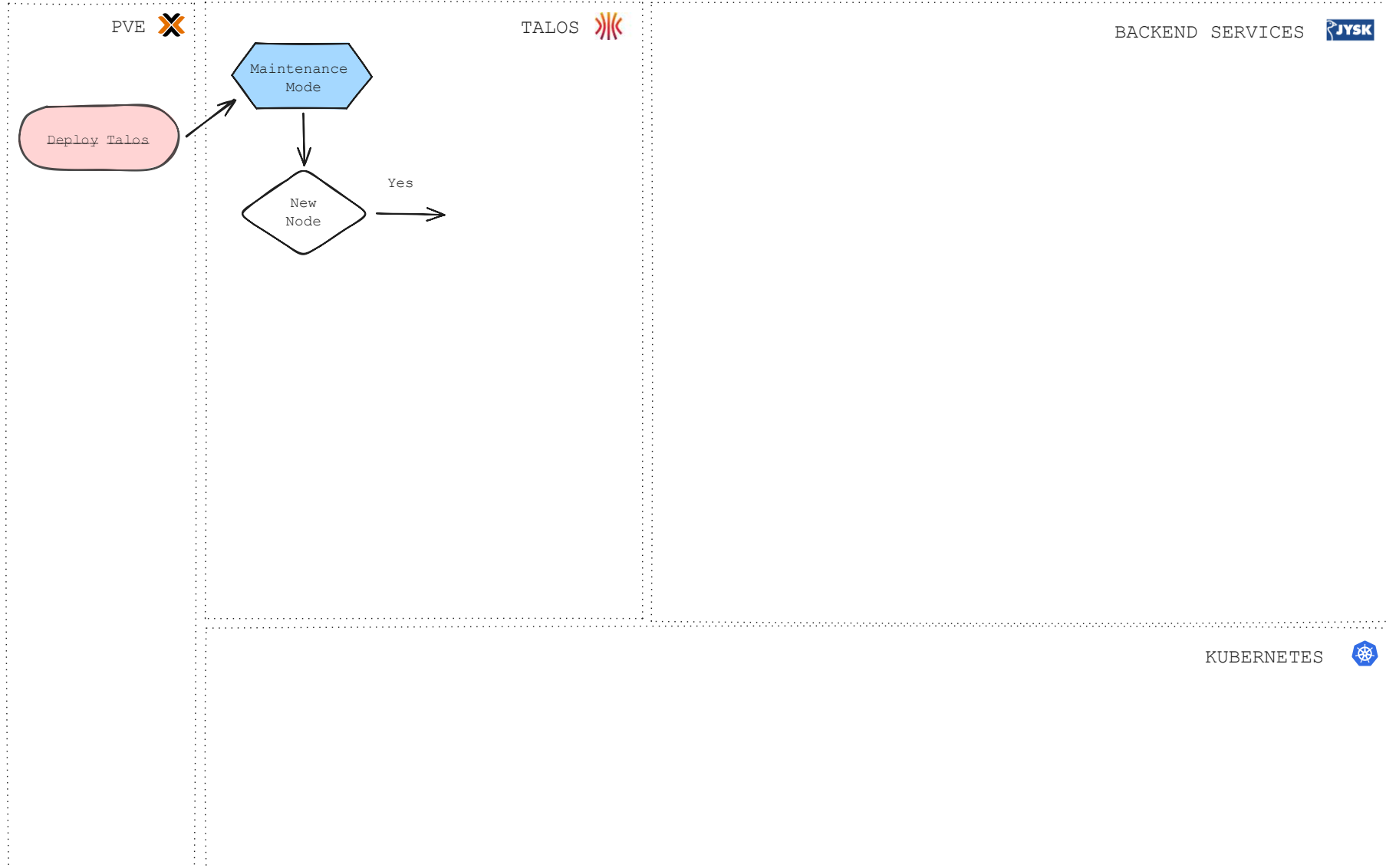
Deployment Process



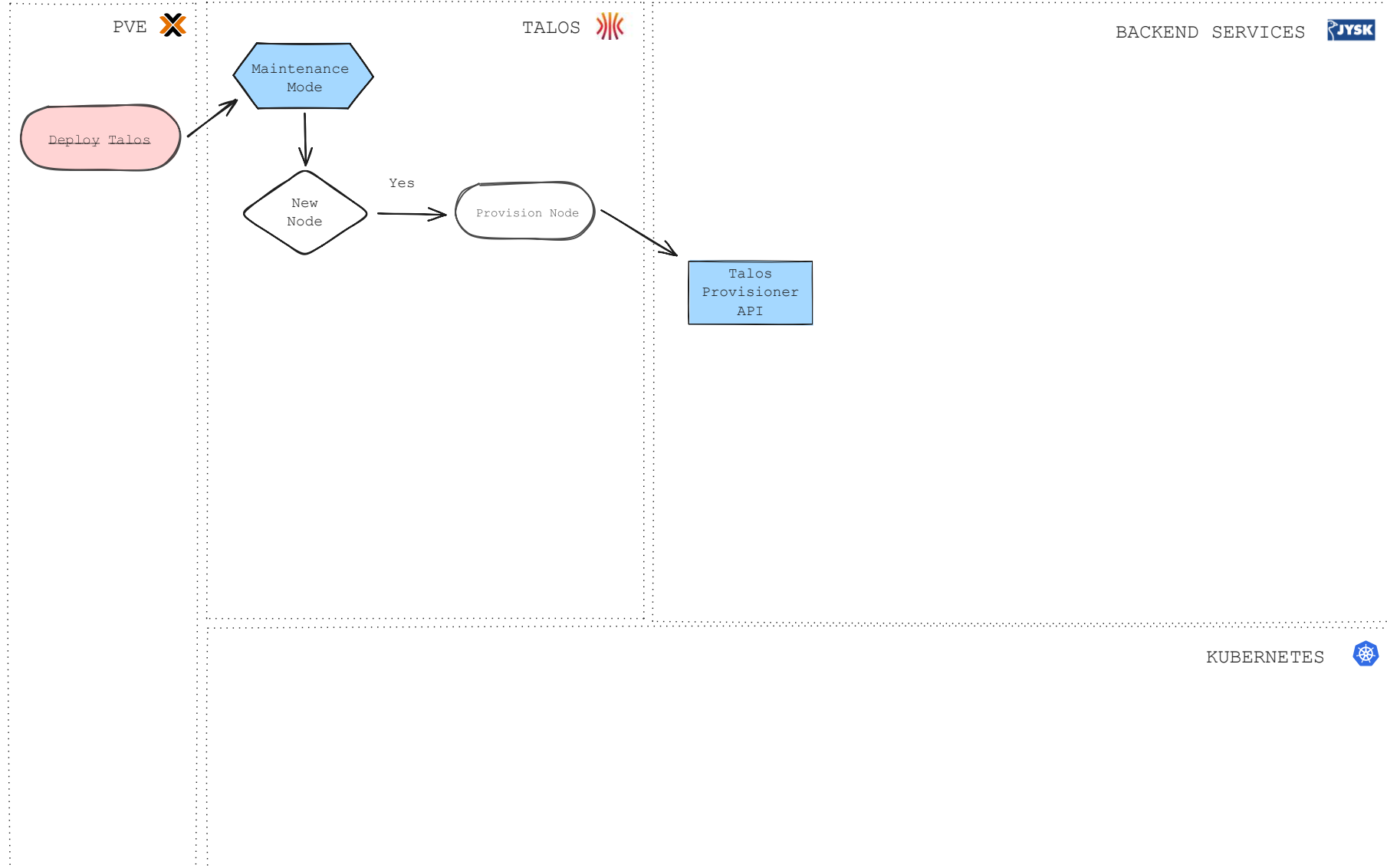
Deployment Process



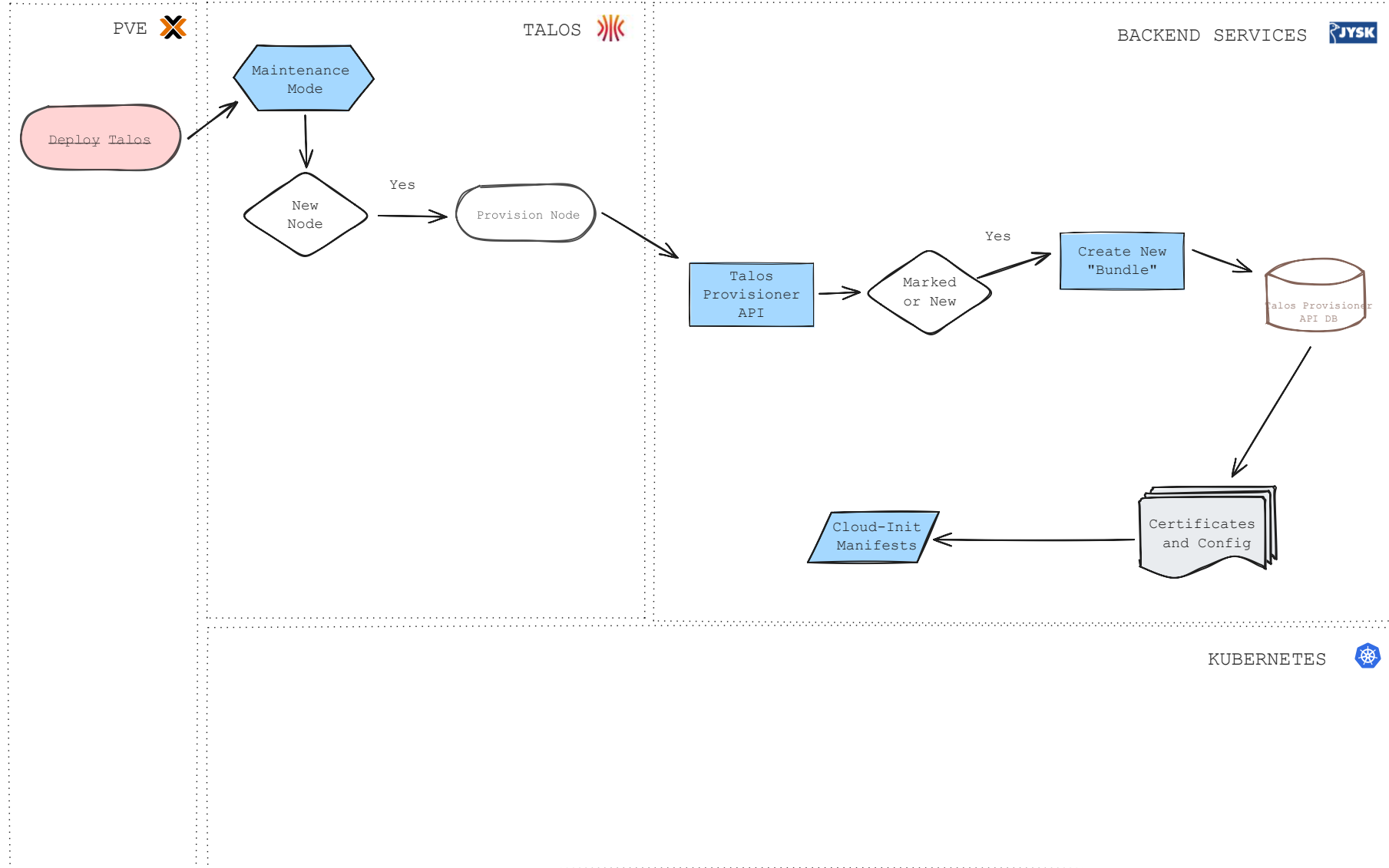
Deployment Process



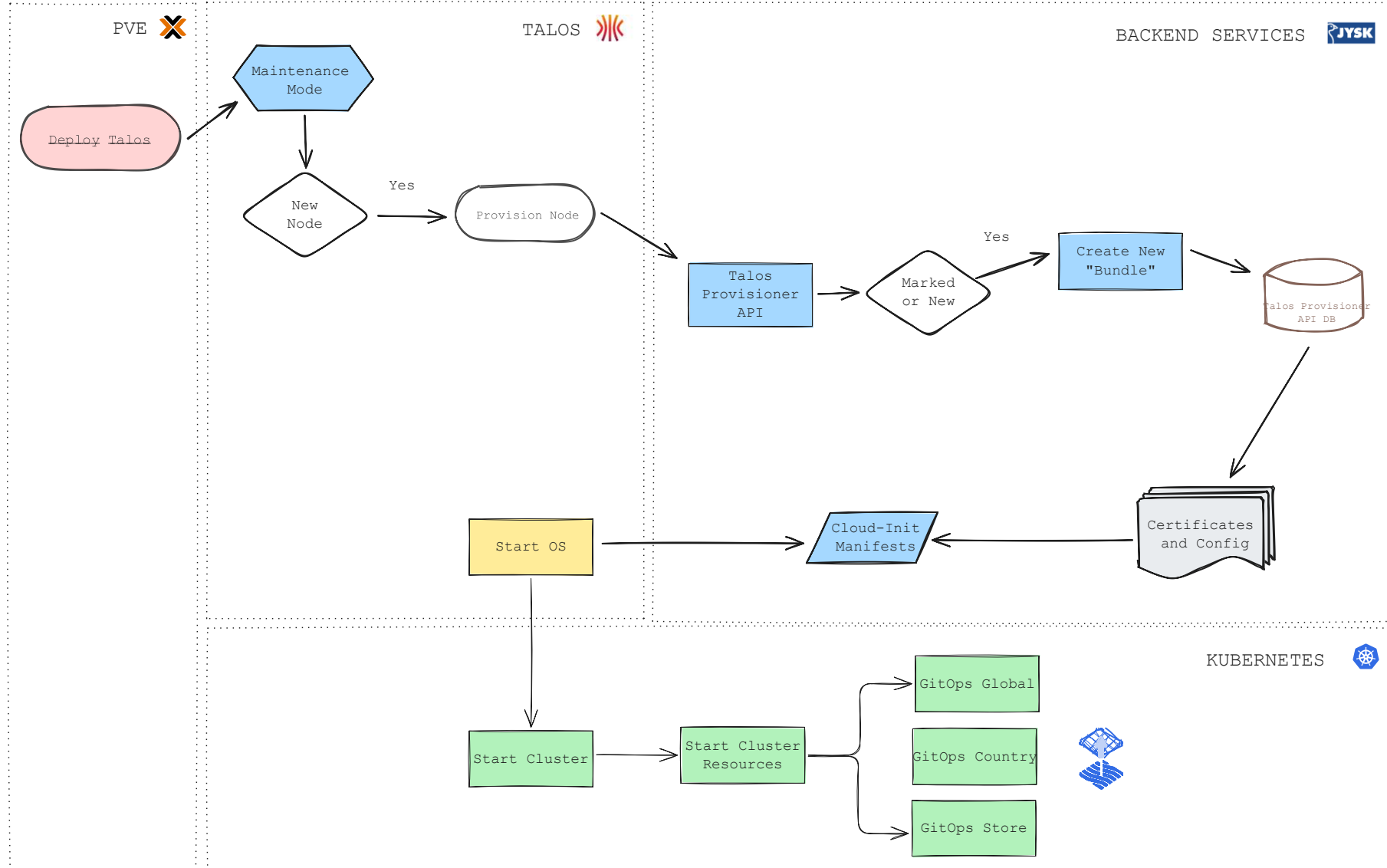
Deployment Process



Deployment Process



Deployment Process

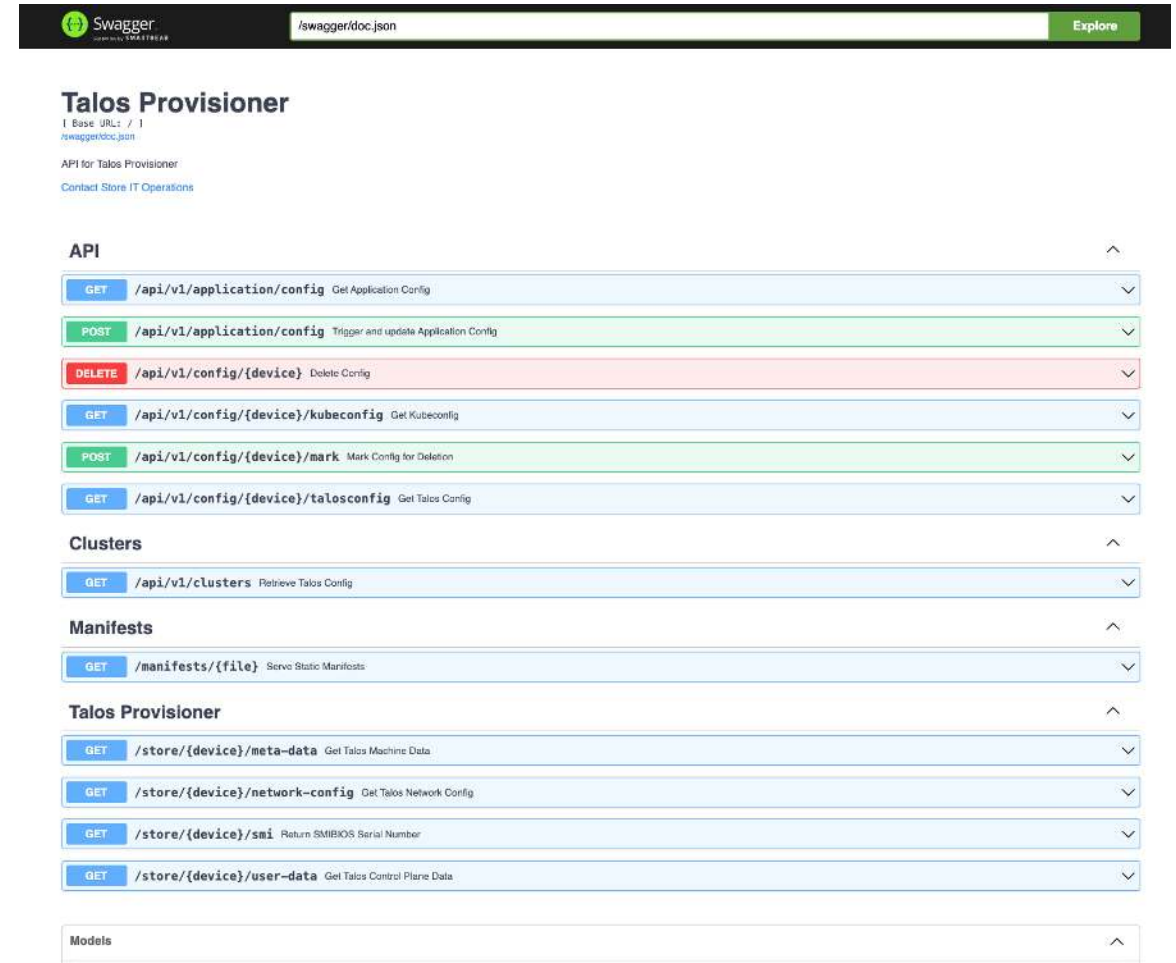




Talos Provisioner API

Custom API and CLI application, An alternative to Omni

- Small API built in Golang
- Uses the Talos upstream SDK
- OAuth2Proxy in front
- Postgres DB for config management
 - Stores Kubeconfig and Talosconfig
- Provides cloudinit manifests for stores
- Generates configs “on-the-fly”

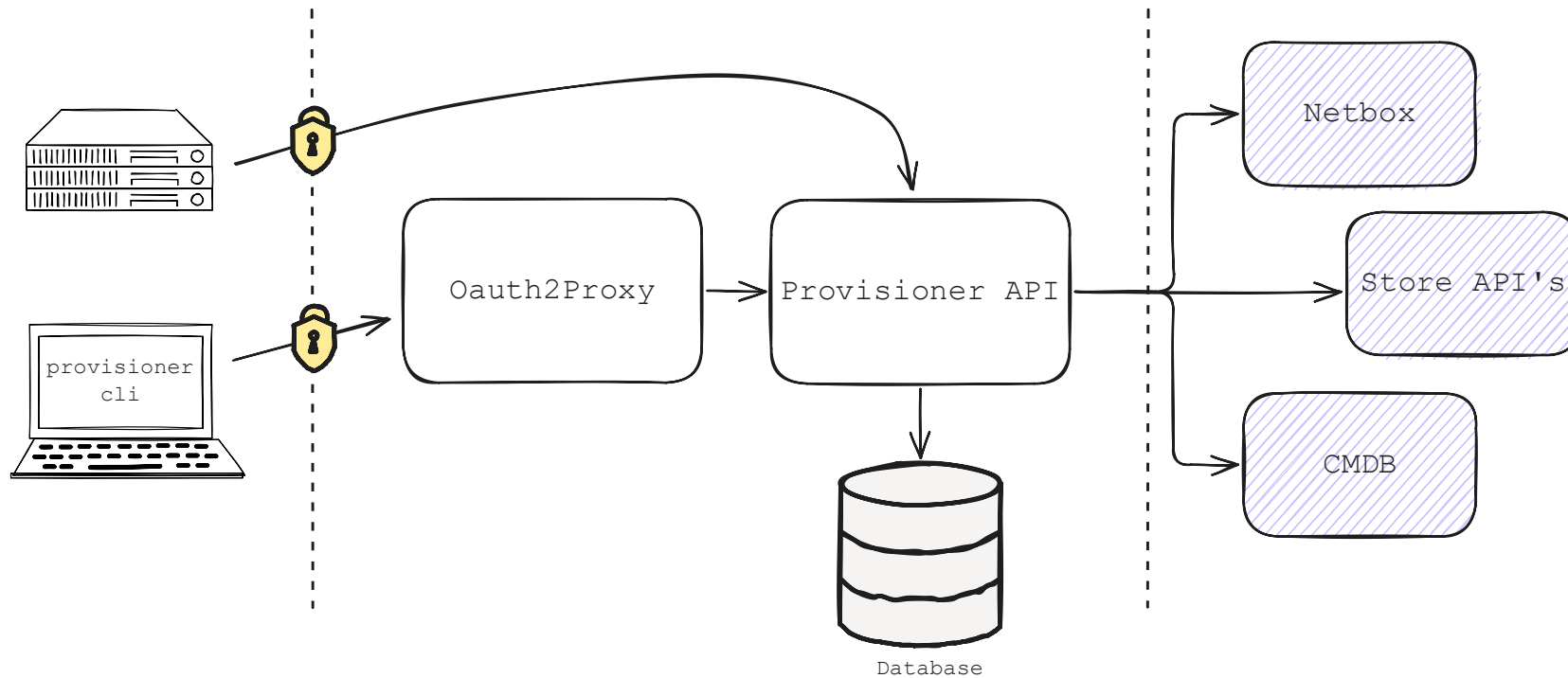


The image shows the Swagger UI for the Talos Provisioner API. At the top, there's a Swagger logo and a search bar containing "/swagger/doc.json" with an "Explore" button. Below this, the title "Talos Provisioner" is displayed, followed by the base URL "Base URL: /" and the Swagger file path "/swagger/doc.json". There are also links for "API for Talos Provisioner" and "Contact Store IT Operations".

The API endpoints are organized into sections:

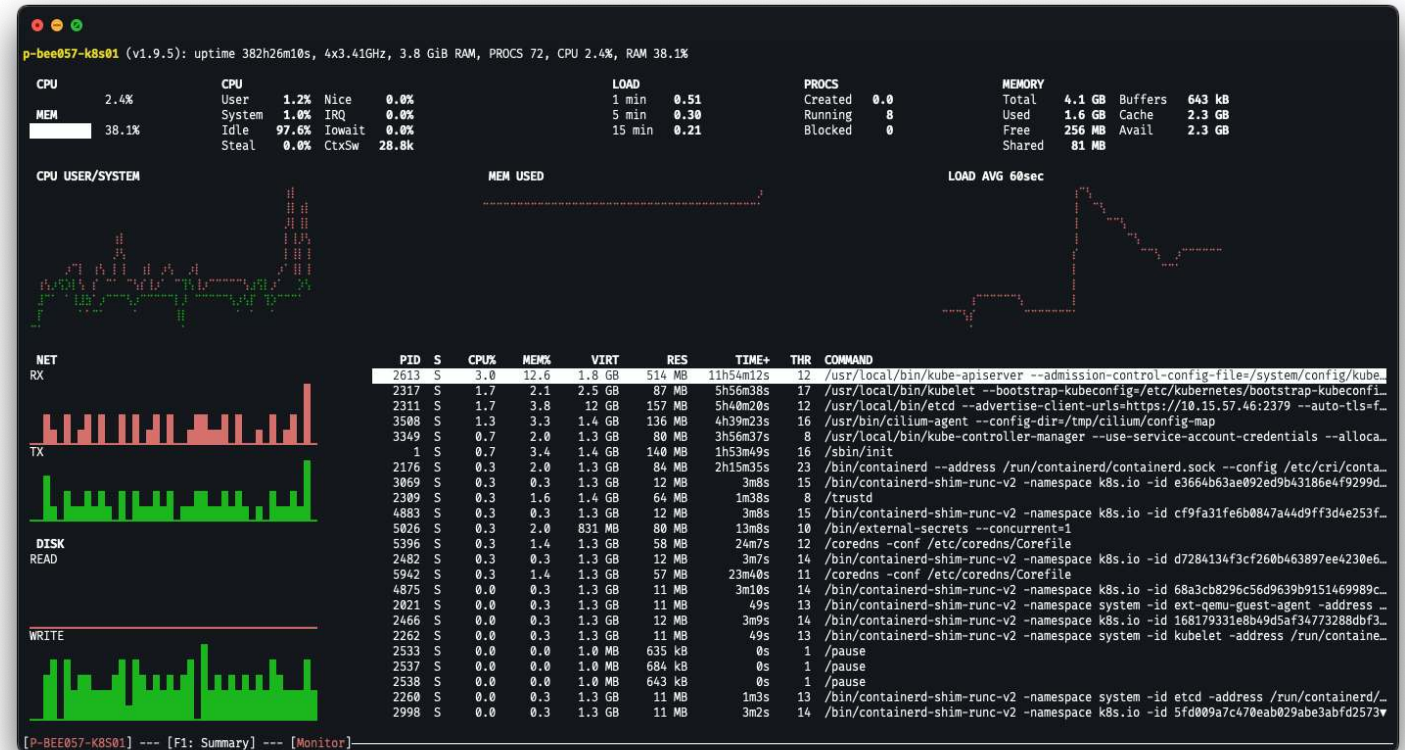
- API**
 - GET /api/v1/application/config: Get Application Config
 - POST /api/v1/application/config: Trigger and update Application Config
 - DELETE /api/v1/config/{device}: Delete Config
 - GET /api/v1/config/{device}/kubeconfig: Get Kubeconfig
 - POST /api/v1/config/{device}/mark: Mark Config for Deletion
 - GET /api/v1/config/{device}/talosconfig: Get Talos Config
- Clusters**
 - GET /api/v1/clusters: Retrieve Talos Config
- Manifests**
 - GET /manifests/{file}: Serve Static Manifests
- Talos Provisioner**
 - GET /store/{device}/meta-data: Get Talos Machine Data
 - GET /store/{device}/network-config: Get Talos Network Config
 - GET /store/{device}/smi: Return SMBIOS Serial Number
 - GET /store/{device}/user-data: Get Talos Control Plane Data
- Models**

Talos Provisioner API



Talos Provisioner CLI

- Built in Golang
- Wrapper around talosctl
- Extra functionality for:
 - CRUD operations for API
 - Authenticating with OIDC
 - Quick launching for k9s



Benefits of a self-built solution

- Built for the purpose of our needs
 - We add custom logic for node creation
 - Delivering certificates and configurations and modifying on the fly
 - Authentication workflows for delivering configuration via CLI
- Very small, and very extensible
 - Can easily interact with other API's in our landscape
 - Can support hooks, notifications etc.
- Free to run (not to maintain!)
- Using the Talos SDK for upstream compatibility (Golang)

Alternatives to a self built solution

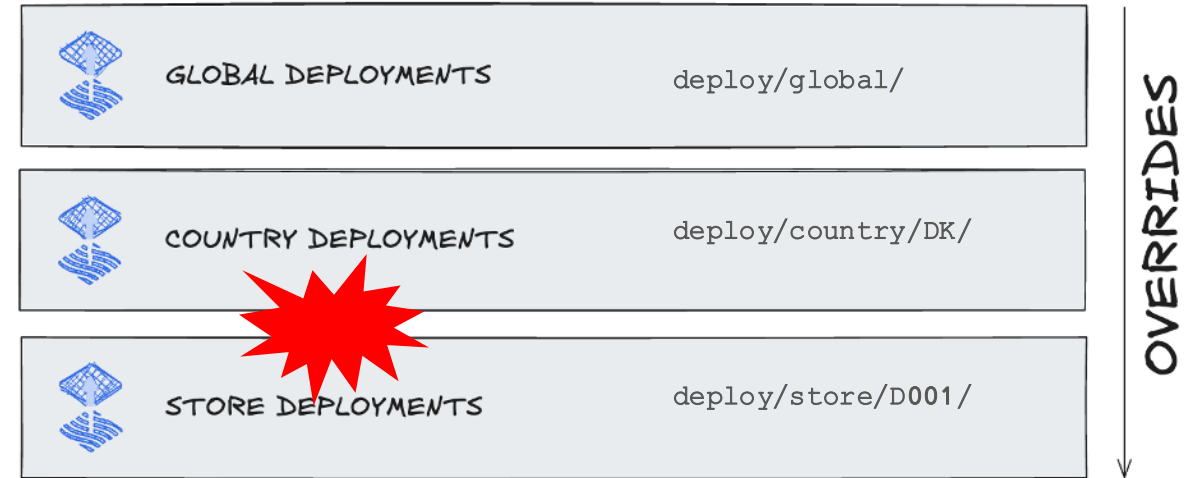
- Nothing that can fully satisfy the requirements yet
- Cluster API can do a lot

Is this enterprise ready?

Issue #1: Scoped Rollouts

Soft Rollouts

- Choose between:
 - "global" deployment,
 - "country" deployments,
 - "store" deployments.
- Talos Provisioner API dynamically configures FluxCD Kustomizations to their respective country, i.e. DK, PL etc.
- **DOESN'T WORK :D**
 - A "race condition"
 - Need to manually intervene
- **NOW FIXED!**



```
➔ ~ k get kustomizations -A
NAMESPACE   NAME                               AGE  READY  STATUS
flux-system  jysk-bootstrap                    93d  True   Applied revision: main@sha1:74ee84006d9a3d3e669bf0ae06978412aa9c2ea1
flux-system  jysk-country-deployments          93d  True   Applied revision: main@sha1:74ee84006d9a3d3e669bf0ae06978412aa9c2ea1
flux-system  jysk-deployments                  93d  True   Applied revision: main@sha1:74ee84006d9a3d3e669bf0ae06978412aa9c2ea1
flux-system  jysk-store-deployments            93d  Unknown Reconciliation in progress
➔ ~
```


Issue #2: Resource Caching

Solves the issues:

- Slow line-speeds being bottlenecks for upgrades

- Avoiding DDoS your own registry and internet-lines

- Caches Talos and Kubernetes images locally
- Manifests can be cached or pulled on boot depending on your needs
- Creates a local registry service

3000+ Clusters Part 4: How not to DDoS your internal registry



Daniel Hansen

Follow

6 min read · Just now



JYSK TECH

Deploying containers in environments with limited internet access or in big scale can be challenging. Talos Linux simplifies this with a local image cache that stores container images on the node itself. By avoiding repeated

Issues

- Scalability should be considered
- Vendor lock-in? Or becoming your **own** vendor?
- Infrastructure limitations
 - Line-speed, caching etc. don't DDoS yourself ☹
 - Vault and Harbor
- Day 2 operations is still not *fully* satisfied
 - Rollout of new upgrades is still managed by another team
 - Rollout is not using preferred methods, we replace nodes.
- Currently only a single node – more work needed for cluster, HA etc.
- System commands are easy with our CLI, but hard at scale
- Utilization of Sidero link (wireguard) in our API?
- Not harvesting system logs
- GitOps is not fully streamlined
 - FluxCD > ArgoCD
 - Global, Country and Store repo's should be split
- Still a long way to go...

Going Forward

- Business has validated the use-case
 - Ideas are being generated
 - Teams are interested
-
- For us "Hybrid" is the way forward
 - Use edge, on-prem, cloud where needed
 - Best use-case



Bare-metal

- Looking into running Talos on Bare-metal
- Omni management plane
- PoC Environment
 - 1 PB of storage
 - 20TB of Memory



Validating Value

Did it work?

- Clear value to the **"customer"**
 - Clear innovative value to **engineering**
 - Clear value to **management**
 - Clear vision for the future
-
- Talks/Presentations
 - On-boarding
 - Tech articles
 - Case studies
 - Testimonials

What Next?

Further info

JYSK.tech has three guides into deployment

- Unleashing the Power of k3s for Edge Computing: Deploying 3000+ in-store Kubernetes Clusters — Part 1
- 3000+ Clusters Part 2: The journey in edge compute with Talos Linux
- 3000+ Clusters Part 3: How to boot Talos Linux nodes with cloud-init and NoCloud



Unleashing the Power of k3s for Edge Computing: Deploying 3000+ in-store...

The initial instalment of our three-part series delves into how JYSK employs Kubernetes across more than 3000 stores!



3000+ Clusters Part 2: The journey in edge compute with Talos Linux

Learn the challenges we face with 3000+ clusters and day 2 ops. Talos, an immutable OS for Kubernetes, simplifying edge compute.



3000+ Clusters Part 3: How to boot Talos Linux nodes with cloud-init and NoCloud

In this article, we dive into a custom built API for generating data for our NoCloud Talos nodes. We'll examine its design, security...

Thank You

dnh@jysk.com / kham@jysk.com